# predict future sales

May 18, 2026

[121]:
```
import numpy as np
import pandas as pd
```

[122]:
```
train_data = pd.read_csv('competitive-data-science-predict-future-sales/
 ↪sales_train.csv')
sale_item = pd.read_csv('competitive-data-science-predict-future-sales/items.
 ↪csv')
item_categories = pd.read_csv('competitive-data-science-predict-future-sales/
 ↪item_categories.csv')
shop_name = pd.read_csv('competitive-data-science-predict-future-sales/shops.
 ↪csv')
test_data = pd.read_csv('competitive-data-science-predict-future-sales/test.
 ↪csv')
```

[123]:
```
train_data
```

[123]:

| | date | date_block_num | shop_id | item_id | item_price | \ |
|---|---|---|---|---|---|---|
| 0 | 02.01.2013 | 0 | 59 | 22154 | 999.00 | |
| 1 | 03.01.2013 | 0 | 25 | 2552 | 899.00 | |
| 2 | 05.01.2013 | 0 | 25 | 2552 | 899.00 | |
| 3 | 06.01.2013 | 0 | 25 | 2554 | 1709.05 | |
| 4 | 15.01.2013 | 0 | 25 | 2555 | 1099.00 | |
| … | … | … | … | … | … | |
| 2935844 | 10.10.2015 | 33 | 25 | 7409 | 299.00 | |
| 2935845 | 09.10.2015 | 33 | 25 | 7460 | 299.00 | |
| 2935846 | 14.10.2015 | 33 | 25 | 7459 | 349.00 | |
| 2935847 | 22.10.2015 | 33 | 25 | 7440 | 299.00 | |
| 2935848 | 03.10.2015 | 33 | 25 | 7460 | 299.00 | |

| | item_cnt_day |
|---|---|
| 0 | 1.0 |
| 1 | 1.0 |
| 2 | -1.0 |
| 3 | 1.0 |
| 4 | 1.0 |
| … | … |
| 2935844 | 1.0 |

```
2935845           1.0
2935846           1.0
2935847           1.0
2935848           1.0

[2935849 rows x 6 columns]
```

[124]:
```
def split_month(columns):
    word= columns.split('.')[1]
    return word

train_data['Month']=train_data['date'].apply(split_month)
```

[125]:
```
train_data
```

[125]:
```
                date  date_block_num  shop_id  item_id  item_price  \
0         02.01.2013               0       59    22154      999.00
1         03.01.2013               0       25     2552      899.00
2         05.01.2013               0       25     2552      899.00
3         06.01.2013               0       25     2554     1709.05
4         15.01.2013               0       25     2555     1099.00
…                …               …        …        …           …
2935844   10.10.2015              33       25     7409      299.00
2935845   09.10.2015              33       25     7460      299.00
2935846   14.10.2015              33       25     7459      349.00
2935847   22.10.2015              33       25     7440      299.00
2935848   03.10.2015              33       25     7460      299.00

          item_cnt_day Month
0                  1.0    01
1                  1.0    01
2                 -1.0    01
3                  1.0    01
4                  1.0    01
…                  …      …
2935844            1.0    10
2935845            1.0    10
2935846            1.0    10
2935847            1.0    10
2935848            1.0    10

[2935849 rows x 7 columns]
```

[126]:
```
def split_year(columns):
    word= columns.split('.')[2]
    return word
```

```
train_data['Year']=train_data['date'].apply(split_year)
```

[127]:
```
train_data
```

[127]:
```
                date  date_block_num  shop_id  item_id  item_price  \
0         02.01.2013               0       59    22154      999.00
1         03.01.2013               0       25     2552      899.00
2         05.01.2013               0       25     2552      899.00
3         06.01.2013               0       25     2554     1709.05
4         15.01.2013               0       25     2555     1099.00
…                …               …       …        …           …
2935844   10.10.2015              33       25     7409      299.00
2935845   09.10.2015              33       25     7460      299.00
2935846   14.10.2015              33       25     7459      349.00
2935847   22.10.2015              33       25     7440      299.00
2935848   03.10.2015              33       25     7460      299.00

          item_cnt_day Month  Year
0                  1.0    01  2013
1                  1.0    01  2013
2                 -1.0    01  2013
3                  1.0    01  2013
4                  1.0    01  2013
…                  …    …    …
2935844            1.0    10  2015
2935845            1.0    10  2015
2935846            1.0    10  2015
2935847            1.0    10  2015
2935848            1.0    10  2015

[2935849 rows x 8 columns]
```

[128]:
```
train_data['Sale']=train_data['item_price']*train_data['item_cnt_day']
```

[129]:
```
sale_item
```

[129]:
```
                                              item_name  item_id  \
0              !              (   .)         D        0
1      !ABBYY FineReader 12 Professional Edition Full…        1
2          ***           (UNV)                      D        2
3        ***          (Univ)                       D        3
4            ***     (   )                         D        4
…                                                   …        …
22165                      2 [PC,          ]    22165
22166            1 :        [         ]    22166
22167          1 :       8 (+CD).      …    22167
22168                                    Little Inu    22168
```

```
22169                          (        )    22169

       item_category_id
0                    40
1                    76
2                    40
3                    40
4                    40
…                   …
22165                31
22166                54
22167                49
22168                62
22169                69

[22170 rows x 3 columns]
```

[130]:
```
item_categories=[]
for i in train_data['item_id']:
    item_categories.append(sale_item['item_category_id'].iloc[i])

train_data['item_categories']=item_categories
```

[131]:
```
train_data
```

[131]:
```
               date  date_block_num  shop_id  item_id  item_price  \
0        02.01.2013               0       59    22154      999.00
1        03.01.2013               0       25     2552      899.00
2        05.01.2013               0       25     2552      899.00
3        06.01.2013               0       25     2554     1709.05
4        15.01.2013               0       25     2555     1099.00
…              …              …       …       …          …
2935844  10.10.2015              33       25     7409      299.00
2935845  09.10.2015              33       25     7460      299.00
2935846  14.10.2015              33       25     7459      349.00
2935847  22.10.2015              33       25     7440      299.00
2935848  03.10.2015              33       25     7460      299.00

         item_cnt_day Month  Year     Sale  item_categories
0                 1.0    01  2013   999.00               37
1                 1.0    01  2013   899.00               58
2                -1.0    01  2013  -899.00               58
3                 1.0    01  2013  1709.05               58
4                 1.0    01  2013  1099.00               56
…                …   …    …      …              …
2935844           1.0    10  2015   299.00               55
2935845           1.0    10  2015   299.00               55
```

```
2935846           1.0    10  2015   349.00              55
2935847           1.0    10  2015   299.00              57
2935848           1.0    10  2015   299.00              55

[2935849 rows x 10 columns]
```

[132]:
```
train_data[ 'item_id_categories']=train_data['item_id'].apply(str) + ',' +␣
 ↪train_data['item_categories'].apply (str)
```

[133]:
```
train_data
```

[133]:
```
                date  date_block_num  shop_id  item_id  item_price  \
0         02.01.2013               0       59    22154      999.00
1         03.01.2013               0       25     2552      899.00
2         05.01.2013               0       25     2552      899.00
3         06.01.2013               0       25     2554     1709.05
4         15.01.2013               0       25     2555     1099.00
…                …               …       …        …           …
2935844   10.10.2015              33       25     7409      299.00
2935845   09.10.2015              33       25     7460      299.00
2935846   14.10.2015              33       25     7459      349.00
2935847   22.10.2015              33       25     7440      299.00
2935848   03.10.2015              33       25     7460      299.00

         item_cnt_day Month  Year     Sale  item_categories item_id_categories
0                 1.0    01  2013   999.00               37          22154,37
1                 1.0    01  2013   899.00               58           2552,58
2                -1.0    01  2013  -899.00               58           2552,58
3                 1.0    01  2013  1709.05               58           2554,58
4                 1.0    01  2013  1099.00               56           2555,56
…                 …    …    …       …                 …                 …
2935844           1.0    10  2015   299.00               55           7409,55
2935845           1.0    10  2015   299.00               55           7460,55
2935846           1.0    10  2015   349.00               55           7459,55
2935847           1.0    10  2015   299.00               57           7440,57
2935848           1.0    10  2015   299.00               55           7460,55

[2935849 rows x 11 columns]
```

[ ]:
```
# 1. Grab your target variable FIRST while it still exists
training_target = train_data['item_cnt_day']

# 2. Drop the unwanted columns safely
columns_to_drop = ['date', 'date_block_num', 'item_price', 'Month', 'Year',␣
 ↪'Sale', 'item_id_categories', 'item_categories', 'item_cnt_day']
existing_drops = [col for col in columns_to_drop if col in train_data.columns]
X_train = train_data.drop(columns=existing_drops)
```

```
# 3. DIAGNOSTIC PRINT: Let's see if we actually have data left!
print("--- Data Check ---")
print(f"Number of rows in features (X_train): {X_train.shape[0]}")
print(f"Number of columns left: {X_train.shape[1]}")
print(f"Columns being used to predict: {list(X_train.columns)}")
```

[ ]:
```
# 3. DIAGNOSTIC PRINT: Let's see if we actually have data left!
print("--- Data Check ---")
print(f"Number of rows in features (X_train): {X_train.shape[0]}")
print(f"Number of columns left: {X_train.shape[1]}")
print(f"Columns being used to predict: {list(X_train.columns)}")
```

[ ]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

# 1. Initialize the Scikit-learn model
# n_estimators is the number of trees. 100 is a good standard.
# max_depth keeps the trees from growing too large and overfitting.
model = RandomForestRegressor(n_estimators=100, max_depth=6, random_state=42,␣
 ↪n_jobs=-1)

# 2. Train the model on your data
print("Training the Random Forest model... (this might take a moment)")
model.fit(train_data, training_target)
print("Training complete!")

# 3. Make predictions on your training data to see how it did
predictions = model.predict(train_data)

# 4. Calculate your error (RMSE is standard for this Kaggle competition)
rmse = np.sqrt(mean_squared_error(training_target, predictions))
print(f"Training RMSE Error: {rmse:.4f}")
```

[ ]:

[ ]: